



문제 해결을 위한 알고리즘 연습

문제 크기가 n 인 배열에 저장된 데이터(정수)들을 모두 합하여 결과를 출력해 봅시다.

🕒 문제 해결 방법

숫자가 저장되어 있는 배열에서 요소의 위치를 나타내는 첨자를 하나씩 다음으로 이동하여 처음부터 끝까지 숫자를 더하면 됩니다.

[입력] 정수들이 저장되어 있는 배열

[출력] 배열 요소들을 모두 합한 값

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	25	87	33	96	26	74	51	84
↓									
결과 : 540									

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 배열의 처음부터 끝까지 다음의 내용을 반복합니다.
 - 1.1 현재 위치의 배열 요소를 누적합니다.
 - 1.2 배열의 다음 요소를 가리키게 합니다.
2. 합을 반환합니다.



🔒 문제 해결의 예

|예| 10개의 요소를 가진 배열 요소들의 합을 구하는 경우

1번째 반복 : 첨자가 0인 요소 값 23을 변수 sum에 누적하면 23이 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 첨자가 1인 요소 값 41을 변수 sum에 누적하면 64가 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 첨자가 2인 요소 값 25를 변수 sum에 누적하면 89가 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 첨자가 3인 요소 값 87을 변수 sum에 누적하면 1760이 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 첨자가 4인 요소 값 33을 변수 sum에 누적하면 209가 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 첨자가 5인 요소 값 96을 변수 sum에 누적하면 305가 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 첨자가 6인 요소 값 26을 변수 sum에 누적하면 3310이 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 첨자가 7인 요소 값 74를 변수 sum에 누적하면 405가 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

9번째 반복 : 첨자가 8인 요소 값 51을 변수 sum에 누적하면 4560이 됩니다.

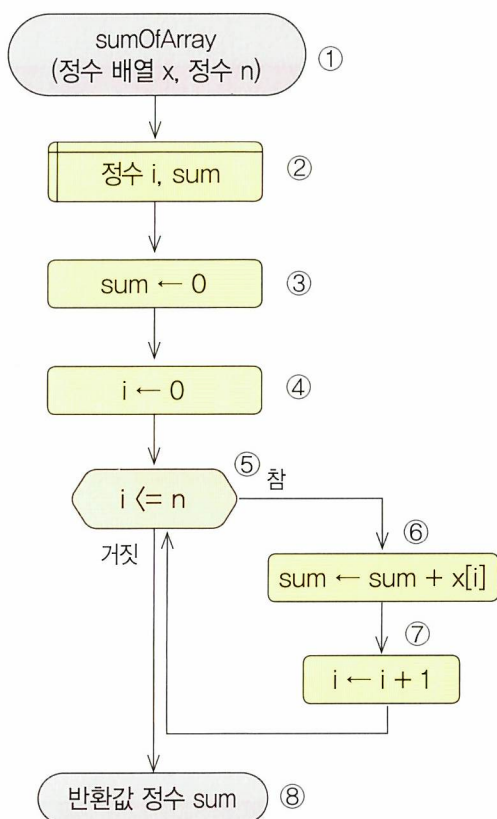
23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

10번째 반복 : 첨자가 9인 요소 값 84를 변수 sum에 누적하면 5400이 됩니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 134쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 sumOfArray입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 n은 배열의 크기를 저장합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하고 배열의 첨자로 사용하기 위한 변수입니다.
- 변수 sum은 배열 요소 값들의 합을 저장하기 위한 변수입니다.

③ 합을 저장하기 위한 변수를 초기화합니다.

- 배열 요소의 값들을 누적하기 전에 변수를 초기화하기 위해 변수 sum에 0을 저장합니다.

④ 반복을 제어하고 배열 요소의 위치를 가리키는 첨자 값을 초기화합니다.

- 처음부터 시작하기 위하여 변수 i의 값을 0으로 초기화합니다.

⑤ 반복 조건을 설정합니다.

- 변수 i의 값이 변수 n의 값보다 작거나 같으면 반복합니다.
즉, 배열의 처음부터 끝까지 반복하도록 합니다.

⑥ 배열 요소의 값을 누적합니다.

- 변수 sum에 배열의 해당 요소의 값을 더합니다.

⑦ 반복 횟수를 제어하는 변수의 값을 변경합니다.

- 변수 i의 값을 1만큼 누적합니다.

⑧ 결과를 반환합니다.

- 배열 요소의 값을 합산한 결과인 변수 sum의 값을 반환합니다.



문제 숫자들이 정렬되어 저장된 배열에 주어진 숫자를 순서에 맞게 삽입해 봅시다.

🕒 문제 해결 방법

주어진 숫자를 정렬되어 있는 배열의 마지막 요소부터 차례로 비교하며, 주어진 숫자보다 크면 비교한 배열 요소를 뒤로 이동시키는 동작을 반복하다가 배열 요소가 주어진 숫자보다 작거나 같아지면 현재의 위치에 주어진 숫자를 저장합니다.

※ 관련 알고리즘 : 2개 숫자를 비교해서 작은 숫자 찾기 (45쪽 참고)

[입력] 숫자들이 정렬되어 저장된 배열, 삽입할 숫자

[결과] 주어진 숫자를 삽입하여 순서대로 삽입된 배열

다음의 예를 통해 내용을 이해해 봅시다.

배열									
17	23	33	41	51	74	84	87	96	
0	1	2	3	4	5	6	7	8	

↓ 26을 삽입									
17	23	26	33	41	51	74	84	87	96
0	1	2	3	4	5	6	7	8	9

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 현재 위치가 배열의 첫 번째 요소가 되거나 배열 요소가 주어진 숫자보다 크지 않을 때까지 다음의 내용을 반복합니다.
 - 현재 배열 요소의 값을 다음 위치로 이동합니다.
 - 앞의 배열 요소를 가리킵니다.
- 주어진 숫자를 현재의 위치에 삽입합니다.
- 주어진 숫자가 삽입된 배열을 반환합니다.



문제 해결의 예

[예] 배열에 26을 추가하는 경우

배열의 처음 상태

17	23	33	41	51	74	84	87	96	
----	----	----	----	----	----	----	----	----	--

96이 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33	41	51	74	84	87		96
----	----	----	----	----	----	----	----	--	----

87이 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33	41	51	74	84		87	96
----	----	----	----	----	----	----	--	----	----

84가 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33	41	51	74		84	87	96
----	----	----	----	----	----	--	----	----	----

74가 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33	41	51		74	84	87	96
----	----	----	----	----	--	----	----	----	----

51이 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33	41		51	74	84	87	96
----	----	----	----	--	----	----	----	----	----

41이 26보다 크므로 한 칸 뒤로 이동합니다.

17	23	33		41	51	74	84	87	96
----	----	----	--	----	----	----	----	----	----

33이 26보다 크므로 한 칸 뒤로 이동합니다.

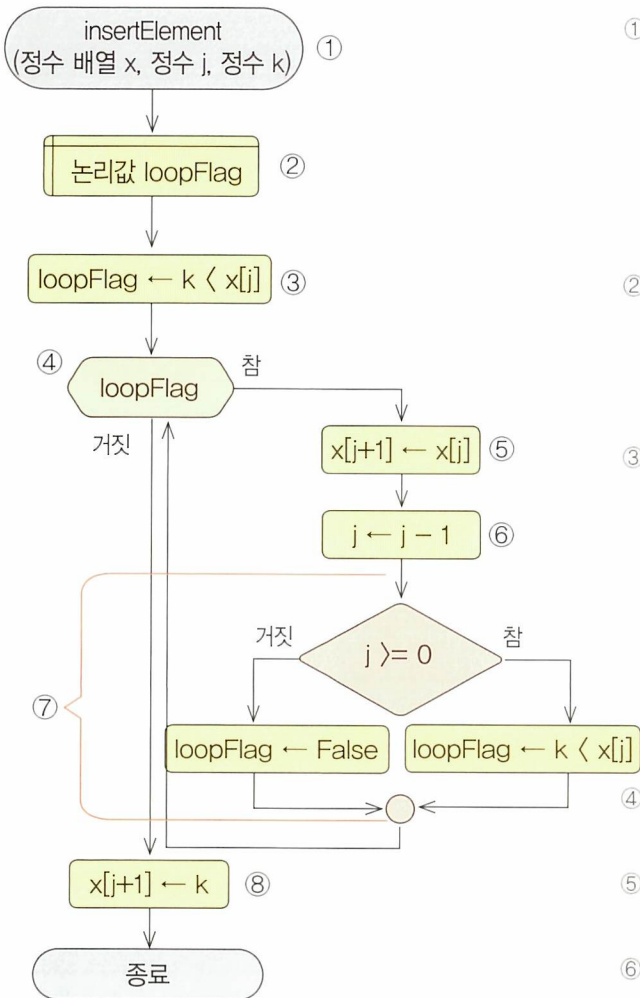
17	23		33	41	51	74	84	87	96
----	----	--	----	----	----	----	----	----	----

배열의 최종 상태 : 23이 26보다 크지 않으므로 현재 위치에 26을 저장합니다.

17	23	26	33	41	51	74	84	87	96
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 135쪽 주프로그램 참고)



- ⑧ 현재 위치의 다음에 삽입하고자 했던 값을 저장합니다.
- 변수 k의 값을 $x[j+1]$ 에 저장합니다.

① 함수를 정의합니다.

- 함수명은 insertElement입니다.
- 배열 x는 정수가 순서대로 저장되어 있는 배열입니다.
- 변수 j은 배열의 마지막 위치(첨자)를 저장합니다.
- 변수 k는 삽입하고자 하는 숫자를 저장합니다.

② 이 알고리즘에서 필요한 논리형 변수는 다음과 같습니다.

- 논리형 변수 loopFlag를 선언하여 반복을 제어합니다.

③ 삽입하는 숫자가 배열의 마지막 요소보다 작을지 비교합니다.

- 비교 결과를 논리형 변수 loopFlag에 저장합니다. k 값이 배열의 마지막 요소보다 작다면 True 값을 저장하여 이어지는 반복 명령을 수행하게 하고, 그렇지 않다면 False 값을 저장하여 반복을 수행하지 않고 열의 마지막에 저장하도록 합니다.

④ 반복 조건을 설정합니다.

- loopFlag의 값이 참이면 반복합니다.

⑤ 현재 위치의 값을 뒤로 이동합니다.

- $x[j]$ 의 값을 $x[j+1]$ 에 저장합니다.

⑥ 현재 위치를 앞으로 이동합니다.

- 변수 j의 값을 1만큼 감소시킵니다.

⑦ 반복 지속 여부를 검사합니다.

- 변수 j의 값이 0보다 크거나 같다면 변수 loopFlag에 변수 k의 값이 현재 위치의 요소보다 작을지에 대한 값을 저장하고, 0보다 작다면 변수 loopFlag에 False를 저장합니다(※ 45쪽 참고).



문제 1부터 n까지 정수의 합을 구해 봅시다.

🕒 문제 해결 방법

$1+2+3+\dots+n$ 을 한꺼번에 더하는 것이 아니라 $1+2$ 를 더해 변수 sum에 저장하고 변수 sum에 저장된 값에 다시 3을 더하는 방법으로 이 문제를 해결합니다. 그러기 위해서는 1부터 n까지 숫자를 변경하기 위한 변수와 숫자들의 합을 저장할 수 있는 변수가 있어야 합니다. 또한, 1부터 n까지 변화하는 것은 횟수 반복을 사용하면 구현할 수 있으며 숫자들의 합을 저장하는 것은 누적 연산 명령을 사용하면 됩니다.

※ 관련 알고리즘 : 순서도 예 - 누적 연산인 경우 (21쪽 참고)

[입력] 정수

[출력] 1부터 정수 n까지의 합

표에 정리된 예를 살펴봅시다.

입력 값	연산 내용	결과 값
5	1부터 5까지의 합	15
10	1부터 10까지의 합	55
100	1부터 100까지의 합	5050

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 1부터 n까지 다음을 반복합니다.
 - 1.1 1만큼씩 증가하는 수를 누적합니다.
2. 합을 반환합니다.



🔒 문제 해결의 예

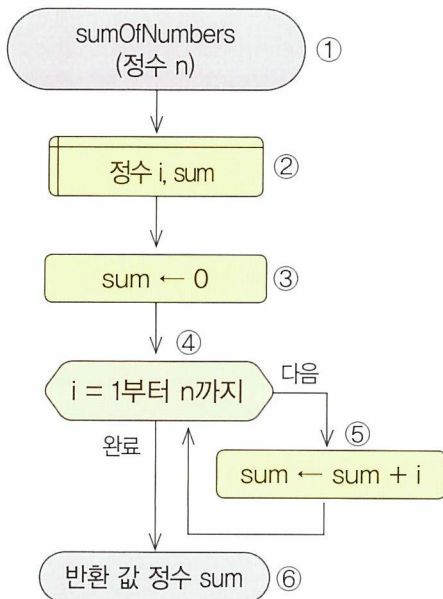
|예| 1부터 10까지의 합

1씩 증가하며
변하는 숫자

1	1
2	3
3	6
4	10
5	15
6	21
7	28
8	36
9	45
10	55

정수들의 합

🔒 순서도와 알고리즘 해설(※ 136쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 sumOfNumbers입니다.
- 변수 n은 정수를 전달받는 매개변수로, 범위를 설정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i, sum는 정수를 저장하기 위한 변수들입니다.

③ 합을 저장하기 위한 변수를 초기화합니다.

- 값들을 누적하기 전에 변수를 초기화하기 위해 변수 sum에 0을 저장합니다.

④ 반복 횟수를 설정합니다.

- i 값을 1부터 n까지 1씩(1인 경우에 생략 가능) 증가시키면서 반복합니다.

⑤ 숫자를 누적합니다.

- 변수 sum에 변수 i의 값을 더하여 변수 sum에 저장합니다.

⑥ 결과를 반환합니다.

- 1부터 n까지 합이 저장된 변수 sum의 값을 반환합니다.



문제 1부터 n까지의 피보나치 수열을 구해 봅시다.

🕒 문제 해결 방법

피보나치 수열은 처음 두 항을 1로 하고, 다음부터는 이전 항과 그 이전의 항을 더해 만드는 수열을 말합니다. 즉, n번째 피보나치 수 $f(n) = f(n-1) + f(n-2)$ 가 됩니다.

따라서, 피연산자를 순환하여 연산하는 알고리즘에서 연산자를 덧셈(+)으로 하여 응용하면 됩니다.

※ 관련 알고리즘 : 피연산자를 순환하여 연산하기 (29쪽 참고)

[입력] 정수

[출력] 1부터 주어진 정수까지의 피보나치 수열

표에 정리된 예를 살펴봅시다.

입력 값	연산 내용	결과 값
10	1부터 10까지의 피보나치 수열	1, 1, 2, 3, 5, 8
30	1부터 30까지의 피보나치 수열	1, 1, 2, 3, 5, 8, 13, 21
50	1부터 50까지의 피보나치 수열	1, 1, 2, 3, 5, 8, 13, 21, 34

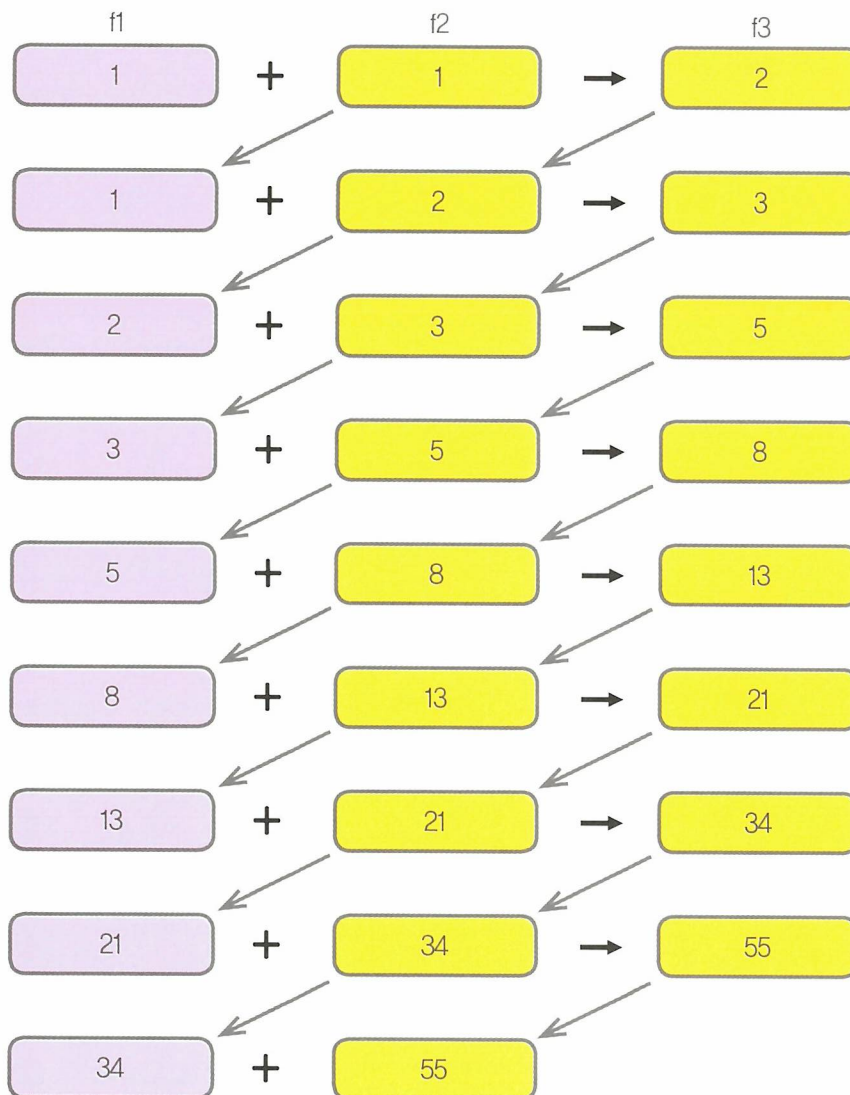
다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 첫 번째와 두 번째 숫자를 초기화합니다.
2. 첫 번째 숫자가 주어진 숫자보다 작은 동안 다음을 반복합니다.
 - 2.1 첫 번째 숫자를 출력합니다.
 - 2.2 첫 번째 숫자와 두 번째 숫자를 더하여 세 번째 숫자를 만듭니다.
 - 2.3 두 번째 숫자를 첫 번째 숫자로 이동합니다.
 - 2.4 세 번째 숫자를 두 번째 숫자로 이동합니다.



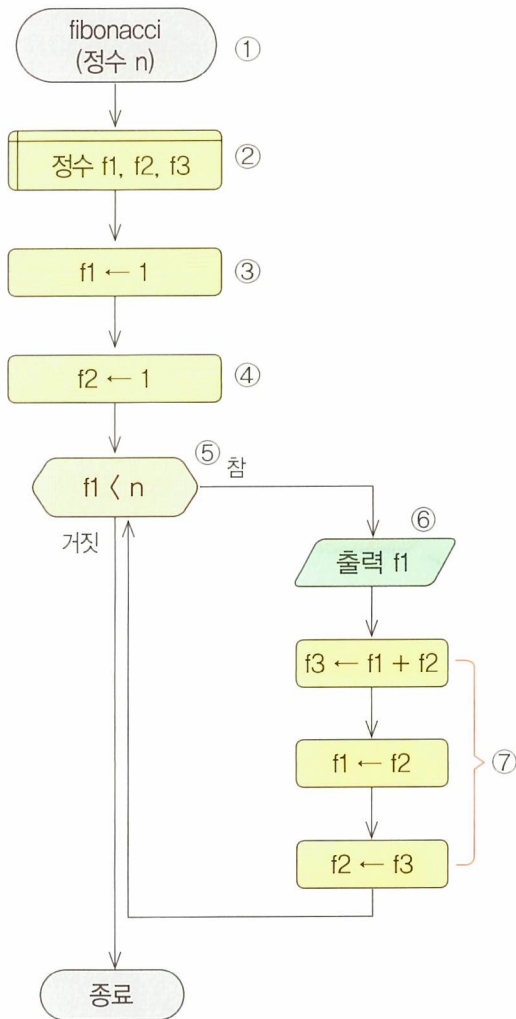
🔒 문제 해결의 예

[예] 1부터 50까지의 피보나치 수열





순서도와 알고리즘 해설(※ 137쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 fibonacci입니다.

- 변수 n은 정수를 전달받는 매개변수로, 입력 받은 값을 저장합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 f1, f2, f3는 정수를 저장하기 위한 변수입니다.

③ 첫 번째 숫자를 초기화합니다.

- 변수 f1에 1을 저장합니다.

④ 두 번째 숫자를 초기화합니다.

- 변수 f2에 1을 저장합니다.

⑤ 반복 조건을 설정합니다.

- f1의 값이 n보다 작은 동안 반복합니다.

⑥ 변수 f1의 값을 출력합니다.

- 수열을 출력합니다.

⑦ 변수들의 값을 갱신합니다.

- 덧셈 연산자를 사용한 피연산자를 순환하여 연산하기(※ 29쪽 참고)를 실행합니다.



문제 주어진 정수의 2배 이상의 배수를 표시해 봅시다(단, 1배수는 제외합니다.).

🕒 문제 해결 방법

1배수는 제외하므로 첫 번째 요소의 첨자는 주어진 숫자에 2를 곱한 수와 일치합니다. 그 다음부터는 주어진 숫자를 계속 더하여 그 값과 일치하는 첨자를 가진 배열의 요소에 1을 저장하는 것을 반복하면 됩니다.

※ 관련 알고리즘 : 2개 숫자를 비교해서 작은 숫자 찾기 (45쪽 참고)

[입력] 정수

[출력] 주어진 정수의 배수가 표시된 배열 (단, 1의 배수에 해당하는 첨자 제외)

다음의 예를 통해 내용을 이해해 봅시다. 배열의 크기가 10이고, 정수 3이 주어졌다면 1배수인 3을 제외하고 첨자가 6, 9에 해당하는 배열의 요소에 다음과 같이 1을 저장합니다.

배열									
0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

▼ 주어진 숫자 : 3

결과									
0	0	0	0	0	0	1	0	0	1
0	1	2	3	4	5	6	7	8	9

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 주어진 정수의 2배수부터 마지막 배열 요소에 도달할 때까지 주어진 정수만큼씩 건너 뛰며 다음을 반복합니다.
 - 해당하는 배열 요소에 1을 저장합니다.
- 주어진 숫자의 배수를 표시한 배열을 반환합니다.

Chapter
4

문제 해결의 예

|예| 19까지 3의 배수 찾기

처음 상태 : 배열의 모든 요소의 값을 0으로 초기화합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1번째 반복 : 3의 2배수인 6이 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

2번째 반복 : 6에 3을 더한 9가 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0

3번째 반복 : 9에 3을 더한 12가 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	0

4번째 반복 : 12에 3을 더한 15가 첨자인 곳에 1을 저장합니다.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	0	0

5번째 반복 : 15에 3을 더한 18이 첨자인 곳에 1을 저장합니다.

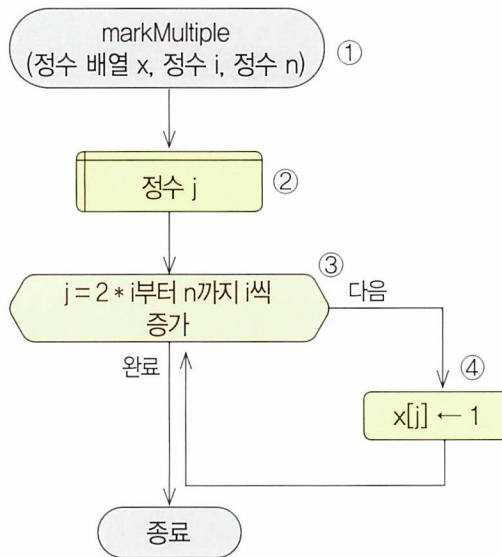
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1	0

결과

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	0	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1	0



 순서도와 알고리즘 해설(※ 138쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 markMultiple입니다.
- 배열 x는 모든 요소에 0이 저장되어 있는 배열입니다.
- 변수 i는 배수를 찾기 위해 주어진 숫자입니다.
- 변수 n은 배열의 범위를 지정합니다(예를 들어, n이 200이면 0부터 20까지에 해당합니다.)

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 j는 반복을 제어하기 위한 변수입니다.

③ 반복 횟수를 설정합니다.

- 변수 i 값의 2배수부터 시작하여 변수 i의 값만큼 증가시키며 반복합니다.

④ 해당 위치에 1을 저장합니다.

- 변수 j의 값과 일치하는 첨자를 가진 배열의 요소에 1로 저장하여 표시합니다.



문제 배열의 요소 값이 주어진 숫자보다 작거나 같은지 표시해 봅시다.

🕒 문제 해결 방법

배열의 각 요소에 저장된 값과 주어진 숫자를 비교하여 배열의 요소가 주어진 숫자보다 작거나 같다면 같은 크기의 또 다른 배열을 만들어 같은 위치에 1을 저장하여 표시하는 것을 반복하면 됩니다(별도의 배열에 표시해야 원래의 숫자들이 지워지지 않습니다.).

[입력] 숫자가 저장되어 있는 배열, 비교할 숫자

[출력] 주어진 숫자보다 작거나 같은 요소와 같은 위치에 1이 표시된 배열

다음의 예를 통해 내용을 이해해 봅시다. 이때, 정수 50이 주어집니다.

배열

23	25	41	33	87	26	74	51	84	96
----	----	----	----	----	----	----	----	----	----

▼ 주어진 숫자 : 50

결과

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0

다음과 같은 순서로 문제를 해결할 수 있습니다.

- 배열의 처음부터 끝까지 다음 내용을 반복합니다.
 - 만약 배열의 요소 값과 주어진 숫자가 작거나 같다면
 - 별도 배열의 같은 위치에 1을 저장합니다.
- 결과를 반환합니다.



🔒 문제 해결의 예

|예| 크기가 10인 배열의 요소와 50을 비교

1번째 반복 : 23이 50보다 작거나 같으므로 별도 배열의 첨자가 0인 곳에 1을 더합니다.

23	25	41	33	87	26	74	51	84	96
1	0	0	0	0	0	0	0	0	0

2번째 반복 : 25가 50보다 작거나 같으므로 별도 배열의 첨자가 1인 곳에 1을 더합니다.

23	25	41	33	87	26	74	51	84	96
1	1	0	0	0	0	0	0	0	0

3번째 반복 : 41이 50보다 작거나 같으므로 별도 배열의 첨자가 2인 곳에 1을 더합니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	0	0	0	0	0	0	0

4번째 반복 : 33이 50보다 작거나 같으므로 별도 배열의 첨자가 3인 곳에 1을 더합니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	0	0	0	0	0

5번째 반복 : 87이 50보다 작거나 같지 않으므로 별도 배열의 첨자가 4인 곳의 값을 그대로 둡니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	0	0	0	0	0

6번째 반복 : 26이 50보다 작거나 같으므로 별도 배열의 첨자가 5인 곳에 1을 더합니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0

7번째 반복 : 74가 50보다 작거나 같지 않으므로 별도 배열의 첨자가 6인 곳의 값을 그대로 둡니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0

8번째 반복 : 51이 50보다 작거나 같지 않으므로 별도 배열의 첨자가 7인 곳의 값을 그대로 둡니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0

9번째 반복 : 84가 50보다 작거나 같지 않으므로 별도 배열의 첨자가 8인 곳의 값을 그대로 둡니다.

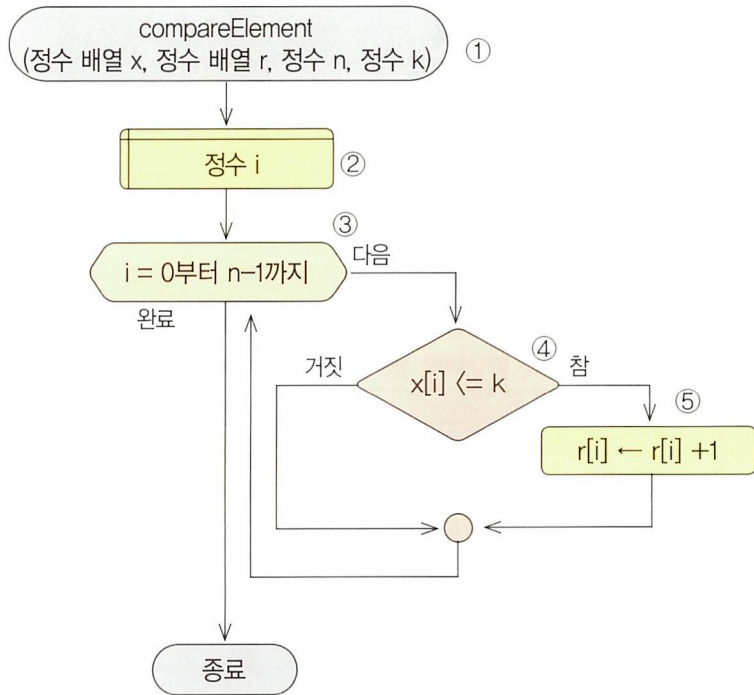
23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0

10번째 반복 : 96이 50보다 작거나 같지 않으므로 별도 배열의 첨자가 9인 곳의 값을 그대로 둡니다.

23	25	41	33	87	26	74	51	84	96
1	1	1	1	0	1	0	0	0	0



순서도와 알고리즘 해설(※ 139쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 compareElement입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 배열 r은 주어진 숫자보다 작거나 같은가를 표시하기 위한 배열로 모든 요소에 0이 저장되어 있습니다.
- 변수 n은 배열의 크기를 저장합니다.
- 변수 k는 크기를 비교하기 위한 기준 숫자가 저장되어 있습니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하기 위한 변수입니다.

③ 반복 횟수를 설정합니다.

- 배열 x의 처음부터 마지막 요소까지 비교하기 위해 0부터 시작하여 n-1까지 1씩 증가시켜 반복합니다.

④ 배열의 요소와 주어진 숫자를 비교합니다.

- 배열 x의 현재 요소가 주어진 숫자 k보다 작거나 같으면 참, 그렇지 않으면 거짓이 됩니다.

⑤ 배열의 요소가 주어진 숫자보다 작거나 같음을 별도 배열에 표시합니다.

- 배열 r의 요소에 배열 x의 해당 요소와 같은 위치에 1을 더합니다(※ 더하지 않고 1을 저장해도 됩니다.).



문제 배열에 있는 숫자들 중에서 가장 작은 숫자가 저장된 위치를 찾아봅시다.

🕒 문제 해결 방법

현재까지 가장 작은 값이 저장된 요소의 위치(첨자)를 저장하고 이 위치의 배열 요소를 다음 요소와 비교합니다. 만약, 현재 가장 작은 값이 저장된 위치의 요소보다 비교한 배열 요소가 작으면 더 작은 값을 가지는 위치를 저장합니다.

※ 관련 알고리즘 : 2개 숫자 비교해서 작은 숫자 찾기 (45쪽 참고)

[입력] 숫자가 저장되어 있는 배열

[출력] 배열에서 작은 값을 가지는 배열 요소의 위치

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	17	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

↓

결과									
24	41	17	87	33	96	26	74	51	84
0	1	2	3	4	5	6	7	8	9

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 배열의 첫 번째 요소 위치를 가장 작은 값을 가지는 요소의 위치로 저장합니다.
2. 배열의 두 번째부터 끝까지 다음을 반복합니다.
 - 2.1 현재 위치의 요소가 가장 작은 값을 가지는 요소보다 더 작다면
 - 2.1.1 현재 위치의 요소를 가장 작은 값을 가지는 요소의 위치로 저장합니다.
3. 가장 작은 값을 가지는 요소의 위치를 반환합니다.

 문제 해결의 예**|예|** 크기가 10인 배열의 요소 중 가장 작은 값 구하기

첫 번째 요소의 위치(첨자)인 0을 변수 min에 저장합니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

1번째 반복 : 41이 23보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 17이 23보다 작으므로 변수 min에 현재 위치(첨자)인 2를 저장합니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 87이 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 33이 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 96이 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 26이 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 74가 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 51이 17보다 작지 않으므로 변수 min의 값이 바뀌지 않습니다.

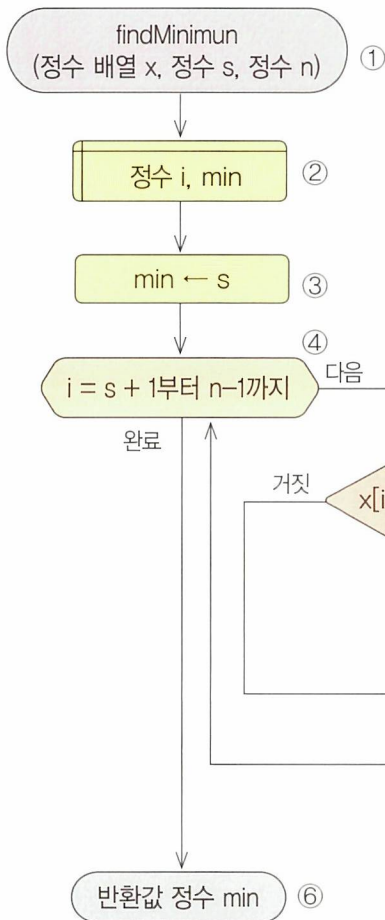
23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

9번째 반복 : 84가 17보다 작지 않으므로 변수 min의 값이 바뀌지 않으며, 최종적으로 2가 됩니다.

23	41	17	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 140쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 findMinimum입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 s는 배열의 시작 위치(첨자)를 저장합니다.
- 변수 n은 배열의 크기를 저장합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하는데 사용합니다.
- 변수 min은 가장 작은 값을 가진 요소의 위치(첨자)를 저장합니다.

③ 배열의 첫 번째 요소 위치를 가장 작은 수의 위치로 저장합니다.

- 주어진 배열의 시작 위치가 변수 s의 값이므로 변수 s의 값을 변수 min에 저장합니다.

④ 반복 횟수를 설정합니다.

- 변수 i의 값을 배열의 2번째 위치부터 1씩 증가시키면서 끝까지 반복합니다.
- 배열의 두 번째 위치는 시작 위치가 변수 s의 값이므로 '변수 s의 값+1'이 됩니다.
- 배열의 끝은 n-1이 됩니다.

⑤ 해당 위치의 값이 현재 가장 작은 값보다 작으면 해당 위치를 저장합니다

- 현재의 배열 요소인 $x[i]$ 가 지금까지의 가장 작은 요소인 $x[\text{min}]$ 보다 작으면 변수 min에 변수 i의 값을 저장합니다(※ 45쪽 참고).

⑥ 가장 작은 숫자가 저장되어 있는 위치를 반환합니다.

- 변수 min의 값을 반환합니다.



[문제] 배열에 저장된 숫자들 중에서 가장 큰 숫자를 배열의 마지막에 저장해 봅시다.

🕒 문제 해결 방법

배열에서 연속되어 있는 두 숫자들을 비교하여 작은 숫자는 앞에, 큰 숫자는 뒤에 나열합니다. 배열의 첫 요소부터 마지막 요소까지 이와 같은 동작을 반복하면 가장 큰 숫자가 배열의 마지막 요소로 오게 됩니다.

※ 관련 알고리즘 : 2개의 숫자를 순서대로 정렬하기 (48쪽 참고)

[입력] 숫자가 저장되어 있는 배열

[출력] 가장 큰 숫자가 마지막에 저장되어 있는 배열

다음의 예를 통해 내용을 이해해 봅시다.

배열									
23	41	25	87	33	96	26	74	51	84
↓									
결과									
23	25	41	33	87	26	74	51	84	96

다음과 같은 순서로 문제를 해결할 수 있습니다.

1. 배열의 처음부터 끝까지 다음을 반복합니다.
 - 1.1 만약 현재 위치의 요소 값이 그 다음 요소 값보다 크다면
 - 1.1.1 두 요소에 저장된 값을 서로 바꿉니다.
2. 결과를 반환합니다.



🔒 문제 해결의 예

|예| 크기가 10인 배열의 요소 중 가장 큰 값을 마지막 요소에 저장하기

1번째 반복 : 23이 41보다 크지 않으므로 그대로 둡니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

2번째 반복 : 41이 25보다 크므로 두 수를 바꿉니다.

23	41	25	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

3번째 반복 : 41이 87보다 크지 않으므로 그대로 둡니다.

23	25	41	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

4번째 반복 : 87이 33보다 크므로 두 수를 바꿉니다.

23	25	41	87	33	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

5번째 반복 : 87이 96보다 크지 않으므로 그대로 둡니다.

23	25	41	33	87	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

6번째 반복 : 96이 26보다 크므로 두 수를 바꿉니다.

23	25	41	33	87	96	26	74	51	84
----	----	----	----	----	----	----	----	----	----

7번째 반복 : 96이 74보다 크므로 두 수를 바꿉니다.

23	25	41	33	87	26	96	74	51	84
----	----	----	----	----	----	----	----	----	----

8번째 반복 : 96이 51보다 크므로 두 수를 바꿉니다.

23	25	41	33	87	26	74	96	51	84
----	----	----	----	----	----	----	----	----	----

9번째 반복 : 96이 84보다 크므로 두 수를 바꿉니다.

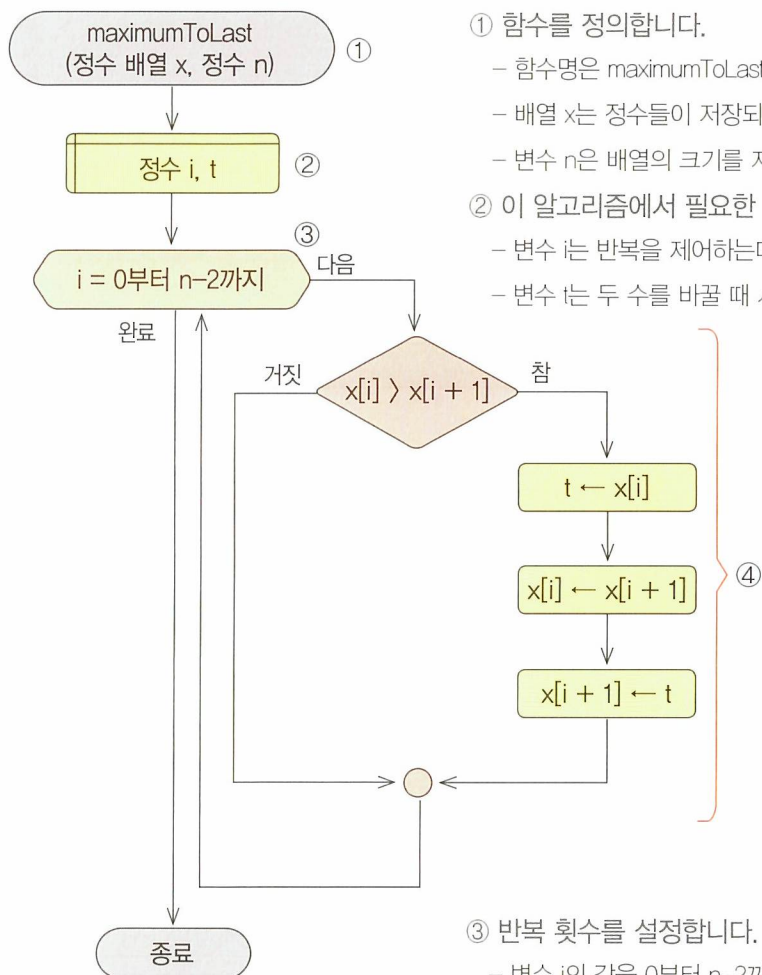
23	25	41	33	87	26	74	51	96	84
----	----	----	----	----	----	----	----	----	----

결과 : 최종적으로 가장 큰 숫자인 96이 마지막에 있음을 확인할 수 있습니다.

23	25	41	33	87	26	74	51	84	96
----	----	----	----	----	----	----	----	----	----



순서도와 알고리즘 해설(※ 141쪽 주프로그램 참고)



① 함수를 정의합니다.

- 함수명은 maximumToLast입니다.
- 배열 x는 정수들이 저장되어 있는 배열입니다.
- 변수 n은 배열의 크기를 지정합니다.

② 이 알고리즘에서 필요한 변수는 다음과 같습니다.

- 변수 i는 반복을 제어하는데 사용합니다.
- 변수 t는 두 수를 바꿀 때 사용합니다.

③ 반복 횟수를 설정합니다.

- 변수 i의 값을 0부터 n-2까지 1씩 증가시키면서 반복합니다.
- 예를 들어, 변수 n의 값이 10이면 0부터 8까지 9번 반복하게 됩니다.

④ 2개의 숫자를 순서대로 나열합니다.

- 변수 i의 값이 가리키는 요소와 뒤에 있는 요소를 비교하여 순서대로 나열합니다(※ 48쪽 참고).